Tecnológico de Monterrey

E2 Parallel syntax highlighter

Juan Pablo Narchi

10 de junio 2025

Implementation of Computational Methods (Gpo 601)

Gilberto Echeverria

Sequential vs. Parallel Performance Analysis

1. Objective: Two Elixir programs were written to colour-code every Python file in a directory.

- **Sequential version** – processes files one after another.
- **Parallel version** – spawns a lightweight task for each file, letting multiple CPU cores work at once.

2. How the Solutions Differ

- **Task scheduling**
  - Sequential: a single linear loop iterates through the file list one by one (the file needs to finish so that the next one can be done).
  - Parallel: each file is handled by its own asynchronous task distributed by the cores of our computer so that we can put color to each file at the same time.
- **IO pattern**
  - Sequential: reads, tokenises, and writes one file completely before starting the next (one by one).
  - Parallel: reading, tokenising, and writing of different files happen concurrently, so disk access often overlaps with CPU work (all at the same time).
- **Overheads**
  - Sequential: Good because none beyond function calls.
  - Parallel: extra task-creation and coordination overhead, but this cost becomes negligible as we give more python files to the program.

3. Analysis of Speed made in my MacBook M1

- **Hardware**: Apple M1, 8-core CPU (fan-less MacBook Air).
- **Software stack**: macOS 14.4 running Erlang/OTP 27 and Elixir 1.17.
- **Datasets**:
  - Set A – three medium Python scripts, like 1,000 lines total.
  - Set B – six scripts like 3,600 lines in total.
- **Procedure**: We analysed the speed of the sequential and parallel program by using the command “time” in my terminal.

4. Results for 3 python scripts

```
--------------------------------
--SEQUENTIAL PROCESSING COMPLETED--
--------------------------------
Processed data_processor.py -> data_processor.html
Processed ml_preprocessor.py -> ml_preprocessor.html
Processed web_scraper.py -> web_scraper.html
elixir lexer.exs    0,37s user 0,24s system 150% cpu 0,404 total
(base) jpnarchi@MacBook-Air-Fausto-Facundo syntax-highlighter % time elixir lexer_pararellism.exs /Users
io/syntax-highlighter/output_files
--------------------------------
--PARARELLISM DONE SUCCESFULLY--
--------------------------------
Processed data_processor.py -> data_processor.html
Processed ml_preprocessor.py -> ml_preprocessor.html
Processed web_scraper.py -> web_scraper.html
elixir lexer_pararellism.exs    0,37s user 0,24s system 167% cpu 0,364 total
```

- **Set A (3 files)**:
  - Sequential average: **0.404 s**
  - Parallel average: **0.364 s**
  - Speed-up: **1.11 ×**

Results for 6 python scripts

```
(base) jpnarchi@MacBook-Air-Fausto-Facundo syntax-highlighter % time elixir lexer.exs /Users/jpnarchi/Desktop/CO
ghlighter/output_files
--------------------------------
--SEQUENTIAL PROCESSING COMPLETED--
--------------------------------
Processed api_client.py -> api_client.html
Processed data_cleaner.py -> data_cleaner.html
Processed file_manager.py -> file_manager.html
Processed ml_pipeline.py -> ml_pipeline.html
Processed text_analyzer.py -> text_analyzer.html
Processed web_crawler.py -> web_crawler.html
elixir lexer.exs    0,46s user 0,23s system 133% cpu 0,521 total
(base) jpnarchi@MacBook-Air-Fausto-Facundo syntax-highlighter % time elixir lexer_pararellism.exs /Users/jpnarch
io/syntax-highlighter/output_files
--------------------------------
--PARARELLISM DONE SUCCESFULLY--
--------------------------------
Processed api_client.py -> api_client.html
Processed data_cleaner.py -> data_cleaner.html
Processed file_manager.py -> file_manager.html
Processed ml_pipeline.py -> ml_pipeline.html
Processed text_analyzer.py -> text_analyzer.html
Processed web_crawler.py -> web_crawler.html
elixir lexer_pararellism.exs    0,50s user 0,26s system 206% cpu 0,369 total
```

- **Set B (6 files)**:
  - Sequential average: **0.521 s**
  - Parallel average: **0.365 s**
  - Speed-up: **1.43 ×**
- **Overall:**
  - Sequential average: 0.521 s
  - Parallel: 0.365 s
  - Average gain: 1.27 ×

5. Complexity Analysis

- Both programs scan every character of all files once, so computational work is proportional to total input size N ($\Theta(N)$), (I think i'm correct, i'm bad identifying this).

- **Sequential wall-time** grows linearly with N.
- **Parallel wall-time** ideally shrinks to Θ(N / C) where C is the number of effective CPU cores, but practical gains are limited by scheduling overhead and disk contention.
- Memory usage is governed by the longest line in a file and is essentially constant in both cases.

The 1.43 × improvement observed for six files matches what is expected when two high-performance cores remain thermally available on the fan-less laptop.

7. Reflection and Ethical Observations

Like, parallelism in computing is pretty cool because it makes us wait less for stuff to load but then it's like your computer starts eating up way more power, which is kinda annoying but there's ways to fix that like batching things together and making cores chill when they're not doing anything. And when you're setting up these big processing systems, you gotta be super careful about all the legal stuff like making sure you're not stealing code or breaking privacy rules and all that boring but important stuff. The accessibility thing is actually really smart because when you have those high-contrast colors in code, it's way easier for people who are new to programming to see what's going on, and it helps people with screen readers too which is really thoughtful. But then there's this whole weird situation where like, these automation tools are great because programmers don't have to do the boring formatting stuff anymore and can work on the fun creative projects, but at the same time it's kinda sad because people whose jobs were basically just reviewing code manually might lose their jobs, which is complicated because technology is supposed to help but sometimes it makes things harder for some people.

8. Conclusions

We tested file-level parallelism and it like made things 1.4 times faster on a regular quad-core laptop when we threw six medium-sized files at it, which is pretty decent. Both ways of doing it are still linear complexity-wise though, so like you're saving time but not actually using less CPU power overall, which makes sense. For future we could try streaming tokens to cut down on per-file overhead, do buffered writes, maybe use DFA-based lexers and even GPU acceleration if we can figure that out.

But overall sometimes I think everything it's useless but when I make the things by my own I get to understand that I can use this knowledge for all my projects (maybe not in elixir professor) but it's very wise to make this kind of implementations in the program to optimize the time regardless of the more use of energy (depending on the implementation you are searching for).